

DATABASE — GateFlow

Versión 2.0 · Evoluciona el diseño de base de datos inicial (Sonnet, sesión previa) según lo definido en 02-ARCHITECTURE.md y 01-BUSINESS_RULES.md. Depende de: 01-BUSINESS_RULES.md, 02-ARCHITECTURE.md · Precede a: 04-API.md

Este documento describe el modelo de datos a nivel conceptual (entidades, campos, relaciones, restricciones e índices necesarios). No contiene sentencias SQL ejecutables — la implementación se traduce a migraciones cuando corresponda, siguiendo lo que aquí se define.

1. Qué se conserva del diseño anterior

El núcleo relacional ya diseñado se mantiene sin cambios de fondo: `tenants`, `roles`, `permisos`, `rol_permisos`, `users`, `user_tenants` (evolución de la idea original de roles separados), `calles`, `manzanas`, `edificios`, `unidades`, `casas`, `departamentos`, `residentes_unidades`, `estados_paquete`, `paquete_historial`, `audit_log`, `notificaciones`. Estas tablas cumplen su propósito tal como fueron diseñadas y no requieren rediseño para soportar los cambios de GateFlow.

Lo que evoluciona son tres cosas: (1) la tabla `paquetes` gana campos nuevos, (2) aparecen entidades nuevas propias de los requisitos de GateFlow (ubicación física jerárquica, catálogos de tamaño/prioridad, incidencias), y (3) se añade infraestructura de soporte que no es una "tabla de negocio" sino mecanismo técnico (generación de código, búsqueda universal, agregación para dashboard, metadatos de sincronización offline).

2. Separación núcleo de plataforma vs. módulo Paquetes

Consistente con la arquitectura modular definida en 02-ARCHITECTURE.md, las tablas se clasifican en dos categorías, y esta clasificación debe respetarse cuando se añadan módulos futuros:

Núcleo de plataforma (compartido por todos los módulos, presente y futuro):

`tenants`, `roles`, `permisos`, `rol_permisos`, `users`, `user_tenants`, `calles`, `manzanas`, `edificios`, `unidades`, `casas`, `departamentos`, `residentes_unidades`, `audit_log`.

Módulo Paquetes (específico de este dominio, no debe ser referenciado directamente por futuros módulos salvo a través de `unidades`):

`estados_paquete`, `paquetes`, `paquete_fotografias`, `paquete_firmas`, `paquete_historial`, `notificaciones`,

`ubicaciones` , `tamanos_paquete` , `prioridades_paquete` , `incidencias` ,
`incidencia_fotografias` , `empresas_paqueteria` , `reservas_codigo_gateflow` .

Cuando se construya el módulo de Visitantes o Accesos, sus tablas (`visitas` , `accesos` , etc.) seguirán este mismo patrón: referencian `unidades` y `users` , **nunca** `paquetes` ni ninguna tabla del módulo de Paquetes.

3. Entidades nuevas de GateFlow

3.1 `ubicaciones` (sustituye al campo simple de "ubicación física")

Propósito: representar la estructura de almacenamiento de cada residencial (racks, niveles, posiciones, lockers, estantes, filas) de forma que cada tenant defina su propia jerarquía sin requerir cambios de esquema.

Diseño: tabla auto-referenciada (jerarquía de profundidad variable), no columnas fijas de "rack/nivel/posición". Cada fila es un nodo de la jerarquía de un tenant.

Campo	Descripción
<code>id</code>	Identificador único del nodo.
<code>tenant_id</code>	Tenant al que pertenece esta ubicación.
<code>padre_id</code>	Nodo padre en la jerarquía (nulo si es raíz, ej. "Rack A" sin padre).
<code>nombre</code>	Nombre visible del nodo (ej. "Rack A", "Nivel 2", "Posición 05", "Locker 12").
<code>tipo_nodo</code>	Etiqueta libre definida por el tenant para el nivel jerárquico (ej. "rack", "nivel", "posicion", "locker", "estante", "fila") — no es un catálogo fijo de la plataforma, es dato de configuración del tenant.
<code>activo</code>	Si el nodo sigue disponible para asignar paquetes.
<code>creado_en</code>	Fecha de creación.

Restricciones: un nodo no puede ser padre de sí mismo, ni generar ciclos en la jerarquía. Una ubicación "hoja" (sin hijos) es la que efectivamente se asigna a un paquete; un nodo intermedio (ej. "Rack A") no debe recibir asignaciones directas si tiene hijos activos.

Índices: por `tenant_id` , y por (`tenant_id` , `padre_id`) para resolver la jerarquía eficientemente.

Justificación de diseño: una jerarquía auto-referenciada evita crear una tabla distinta por

cada posible estructura ("racks", "lockers", "estantes") y evita columnas condicionales en paquetes . Es el patrón correcto cuando la profundidad y nomenclatura de la estructura varía por cliente, que es exactamente el requisito (BR-18).

3.2 tamanos_paquete y prioridades_paquete (catálogos configurables)

Propósito: catálogos con valores por defecto provistos por GateFlow (Pequeño/Mediano/Grande/Extra Grande; Normal/Urgente/Medicamento/Documento/Perecedero/Confidencial), extensibles por cada tenant sin afectar a otros (BR-39).

Campo	Descripción
id	Identificador único.
tenant_id	Nulo si es un valor global de la plataforma; asignado si es un valor propio de un tenant.
clave	Identificador corto y estable (ej. urgente , medicamento).
nombre	Nombre visible.
color_hex	Para representación visual (ej. resaltar "Medicamento" en rojo en los listados, BR-19).
orden	Orden de presentación.
activo	Disponibilidad del valor.

Restricción: unicidad de clave dentro del alcance correspondiente (global, o por tenant).

3.3 incidencias

Propósito: módulo de seguimiento de problemas asociados a un paquete, con ciclo de vida independiente del paquete mismo (BR-21 a BR-25).

Campo	Descripción
id	Identificador único.
tenant_id	Tenant al que pertenece (denormalizado para RLS directo).
paquete_id	Paquete al que está asociada. Obligatorio — no existen incidencias sin paquete.

tipo	Catálogo de tipo de incidencia (dañado, abierto, mojado, etiqueta ilegible, destinatario desconocido, rechazado, devuelto, extraviado).
descripcion	Notas libres del guardia o administrador.
estado	Ciclo de vida propio de la incidencia (abierta, en_seguimiento, resuelta) — independiente del estado del paquete.
reportada_por	Usuario que la registró.
resuelta_por	Usuario que la cerró, si aplica.
creada_en / resuelta_en	Fechas del ciclo de vida de la incidencia.

Índices: por (tenant_id, paquete_id) y por (tenant_id, estado) para listados de incidencias abiertas en el dashboard.

3.4 incidencia_fotografias

Propósito: evidencia fotográfica propia de una incidencia, independiente de las fotografías del paquete (BR-24).

Campo	Descripción
id	Identificador único.
incidencia_id	Incidencia a la que pertenece.
storage_path	Ruta del archivo en almacenamiento.
tomada_por	Usuario que la capturó.
creada_en	Fecha de captura.

4. Cambios sobre la tabla `paquetes`

Se añaden los siguientes campos al diseño ya existente de `paquetes`, sin alterar los campos previamente definidos (`unidad_id`, `empresa_paqueteria_id`, `estado_id`, `recibido_por`, `fecha_recepcion`, etc.):

Campo nuevo	Descripción	Justificación
	Código único global con	Identificador propio de GateFlow,

codigo_gateflow	formato GF-AAAA-NNNNNNN .	independiente del courier (BR-11, BR-12).
ubicacion_id	Referencia a un nodo hoja de ubicaciones .	Permite indicarle al guardia dónde está el paquete (BR-17).
tamano_id	Referencia a tamanos_paquete .	Clasificación configurable (BR-19).
prioridad_id	Referencia a prioridades_paquete , opcional.	Clasificación configurable (BR-19).
dispositivo_id	Identificador del dispositivo que originó el registro.	Necesario para el mecanismo de generación de código sin conexión (sección 6) y para trazabilidad offline.
creado_en_dispositivo	Timestamp real de creación, tomado del dispositivo, no del servidor.	Requisito de sincronización offline (BR-35).
sincronizado_en	Timestamp de cuándo el registro llegó efectivamente al servidor.	Permite distinguir demora de sincronización de demora operativa real, y alimenta KPIs de uso del sistema.

Restricción añadida: `ubicacion_id` no puede ser nulo cuando `estado_id = 'recibido'` (aplicación de BR-17 a nivel de restricción de datos, no solo de lógica de aplicación).

5. Búsqueda universal

Decisión de diseño: no se crea un motor de búsqueda externo. Se añade una columna de vector de búsqueda de texto completo (tipo `tsvector` , nativo de Postgres) sobre una vista que combina los campos relevantes de `paquetes` junto con los datos de la unidad y del residente asociado (nombre, teléfono, WhatsApp, tracking, código GateFlow, empresa, estado, fecha).

Esta vista de búsqueda:

- Se filtra siempre por `tenant_id` antes de aplicar el término de búsqueda — el aislamiento de RLS aplica igual que en cualquier otra consulta.
- Se mantiene actualizada mediante un mecanismo de actualización incremental (trigger o

recomputo ligero) cada vez que cambian los datos fuente, no mediante recálculo completo constante.

- Resuelve el requisito de “un único buscador” (BR-42) a nivel de datos: la aplicación nunca necesita decidir en qué tabla buscar, siempre consulta esta vista unificada.

6. Generación de código GateFlow sin conexión

Problema a resolver: el código `GF-AAAA-NNNNNNN` debe ser único globalmente, pero un guardia puede registrar paquetes sin conexión, potencialmente al mismo tiempo que otro guardia en otro residencial.

Mecanismo (tabla `reservas_codigo_gateflow`): al iniciar sesión con conectividad (inicio de turno), cada dispositivo solicita y recibe un bloque reservado de identificadores numéricos (ej. 200 números) para el año en curso. El dispositivo consume ese bloque localmente para generar códigos mientras esté sin conexión, sin necesidad de consultar al servidor por cada paquete. Al recuperar conectividad, el dispositivo solicita un nuevo bloque si el anterior se agota, y nunca hay colisión porque cada bloque se entrega una sola vez a un único dispositivo.

Campo	Descripción
id	Identificador único de la reserva.
dispositivo_id	Dispositivo al que se asignó el bloque.
anio	Año al que corresponde el bloque (coincide con el prefijo <code>AAAA</code> del código).
rango_inicio / rango_fin	Límites numéricos del bloque reservado.
consumido_hasta	Último número efectivamente usado dentro del bloque, para saber cuánto queda disponible.
asignado_en	Fecha de asignación del bloque.

Esta tabla es infraestructura técnica, no una entidad de negocio — no aparece en reportes ni dashboards, existe únicamente para sostener la generación offline de códigos únicos.

7. Dashboard ejecutivo — capa de agregación

Decisión de diseño: las métricas del dashboard (paquetes de hoy, pendientes, entregados,

promedio diario, tiempo promedio hasta entrega, olvidados, top empresas, gráfica de 30 días, incidencias, actividad reciente) no se calculan con consultas en vivo sobre `paquetes` en cada carga de pantalla. Se resuelven con una **vista materializada de métricas diarias por tenant**, recalculada periódicamente (cada pocos minutos, vía tarea programada), que pre-agrega:

- Conteos por estado, por día, por tenant.
- Tiempo promedio entre `fecha_recepcion` y `fecha_entrega` , por día y por tenant.
- Conteo de paquetes que exceden el umbral de "olvidado" configurado por el tenant.
- Conteo de paquetes por empresa de paquetería, por período.
- Conteo de incidencias abiertas.

El dashboard lee exclusivamente de esta capa agregada, nunca de la tabla transaccional directamente — esto es lo que garantiza que el crecimiento histórico de un tenant grande no afecte el tiempo de respuesta del dashboard de otro.

8. Preparación para OCR/IA futura

Se añaden campos opcionales, hoy sin uso funcional, a `paquete_fotografias` :

Campo nuevo	Descripción
<code>ocr_procesado</code>	Booleano, falso por defecto. Indica si esta foto ya fue enviada a un proceso de extracción automática.
<code>ocr_datos_extraidos</code>	Campo estructurado (JSON), nulo hasta que exista el servicio de OCR. Guardará nombre, dirección, tracking, courier y número de guía sugeridos por el proceso automático, sin sobrescribir lo que el guardia haya capturado manualmente.

Ningún flujo actual depende de estos campos; existen para que, cuando se construya el servicio de OCR, no sea necesaria una migración de esquema — solo empezar a poblar columnas que ya existen.

9. Metadatos de sincronización offline (aplican también a fotos y firmas)

`paquete_fotografias` y `paquete_firmas` reciben los mismos campos de trazabilidad offline que `paquetes`: `dispositivo_id`, `creado_en_dispositivo`, `sincronizado_en`. Esto

permite auditar con precisión cuánto tiempo tardó una evidencia en llegar al servidor y detectar patrones de mala conectividad por residencial (útil para decisiones comerciales, ej. recomendar mejorar el internet de la caseta).

10. Estrategia de particionamiento (diferida, no inmediata)

Las tablas de mayor crecimiento esperado — `paquetes`, `audit_log`, `paquete_historial`, `incidencias` — se diseñan desde ahora con `tenant_id` como primer componente de todo índice compuesto relevante, precisamente para que el particionamiento nativo de Postgres por rango de fecha pueda activarse más adelante sin tocar la capa de aplicación. Esto no se implementa en el MVP; se implementa cuando el volumen real (no proyectado) lo justifique, consistente con el principio de costo proporcional al uso definido en `02-ARCHITECTURE.md`.

11. Relación con módulos futuros

Cuando se diseñe el módulo de Visitantes (candidato más probable para V3.0 según `06-ROADMAP.md`), su tabla principal (`visitas`) seguirá el mismo patrón ya establecido aquí: referenciará `unidades` para saber a qué domicilio visita una persona, y `users / user_tenants` para identificar quién la registra y bajo qué rol, sin necesitar tocar ninguna tabla del módulo Paquetes. La tabla `ubicaciones` diseñada en este documento, aunque nace para paquetes, es genérica en su modelo (jerarquía por tenant) y podría reutilizarse sin cambios si un módulo futuro (ej. Vehículos, para cajones de estacionamiento) necesita una estructura de ubicación física similar — esa reutilización se evaluará en su momento, no se fuerza ahora.